

Bài tập về Mẫu thiết kế

Bài 1: Quản lý cấu hình ứng dụng

Xây dựng lớp `AppConfig` quản lý cấu hình ứng dụng.

Yêu cầu:

1. `AppConfig` chỉ có **một** thể hiện duy nhất trong toàn chương trình.
2. Thuộc tính mẫu: `appName`, `version`, `logLevel`.
3. Cung cấp `getInstance()` theo kiểu khởi tạo lười (lazy).
4. Đảm bảo **an toàn đa luồng**.
5. Viết `main` tạo 2 luồng, mỗi luồng in `hashCode()` của `AppConfig` để kiểm tra chỉ có một đối tượng.

Bài 2: Hệ thống gửi thông báo

Thiết kế hệ thống gửi thông báo hỗ trợ nhiều kênh.

Yêu cầu:

1. Tạo giao diện `Notification` với phương thức `send(String msg)`.
2. Có ít nhất 2 loại: `EmailNotification`, `SmsNotification`.
3. Tạo lớp trừu tượng `NotificationApp` có phương thức:
 - `notifyUser(String msg)` (không tạo trực tiếp đối tượng)
 - `createNotification()` (abstract factory method, trả về `Notification`)
4. Tạo các lớp con `EmailApp`, `SmsApp` để quyết định loại `Notification`.
5. `main` chọn một ứng dụng cụ thể và gọi `notifyUser`.

Bài 3: Bộ giao diện người dùng

Thiết kế bộ giao diện gồm hai thành phần: **Button** và **Checkbox**.

Yêu cầu:

1. Tạo interface **Button** và **Checkbox** với phương thức **render()**.
2. Tạo interface **UIFactory** có:
 - **createButton()**
 - **createCheckbox()**
3. Cài đặt ít nhất 2 dòng sản phẩm: **WindowsFactory**, **MacFactory**:
 - **WindowsButton**, **WindowsCheckbox**
 - **MacButton**, **MacCheckbox**
4. **main** nhận tham số cấu hình (ví dụ **"win"** hoặc **"mac"**) để chọn factory, sau đó tạo và render các thành phần.

Bài 4: Adapter + Prototype

(a) Adapter

Hệ thống của bạn yêu cầu interface:

```
interface Sorter {  
    int[] sort(int[] arr);  
}
```

Bạn có thư viện cũ không thể sửa:

```
class LegacySorter {  
    public int[] quickSort(int[] arr) { ... }  
}
```

Yêu cầu:

1. Tạo **SorterAdapter** để dùng **LegacySorter** với **Sorter**.
2. **main** gọi **Sorter** để sắp xếp mảng và in kết quả.

(b) Prototype

Xây dựng lớp `ReportTemplate`

Yêu cầu:

1. Lớp `ReportTemplate` gồm:
 - `title` (String)
 - `footer` (String)
 - `sections` (List<String>)
2. Cài đặt sao chép (clone) để tạo bản sao từ template.
3. Trong `main`, tạo một template gốc và sinh ra 2 bản sao, chỉnh sửa tiêu đề mỗi bản sao khác nhau.
4. In ra 3 báo cáo để kiểm tra template gốc không bị thay đổi.

Bài 5: Chọn và áp dụng mẫu thiết kế phù hợp

Cho các yêu cầu sau, hãy chọn mẫu thiết kế phù hợp trong các mẫu đã học (Singleton, Factory Method, Abstract Factory, Adapter, Prototype) và cài đặt chương trình.

- Mỗi yêu cầu phải dùng đúng Design Pattern đã học
- Viết main để kiểm tra

Yêu cầu:

1. Hệ thống cần một lớp Logger chỉ có một đối tượng duy nhất trong chương trình.
2. Hệ thống cần tạo các đối tượng Export:
 - PdfExport
 - ExcelExport

Việc tạo đối tượng không được viết trực tiếp bằng new trong main.

3. Hệ thống có lớp cũ:

```
class OldPlayer {  
    void playFile(String name) { }  
}
```

Hệ thống mới yêu cầu:

```
interface Player {  
    void play(String name);  
}
```

Không được sửa lớp cũ.

4. Hệ thống cần tạo bản sao của một đối tượng cấu hình để chỉnh sửa mà không làm thay đổi bản gốc.

Bài tập về Mẫu thiết kế (tiếp)

Bài 1: Quản lý hệ thống file đơn giản

Xây dựng công cụ quản lý hệ thống file với ba loại phần tử: `FileItem`, `Shortcut`, `Folder`.

Yêu cầu:

6. Tạo interface/abstract class `FileSystemItem` có phương thức `print(String indent)`.
7. `FileItem`: có `name`, `size(KB)`. Khi in: `File: <name> (<size>KB)`.
8. `Shortcut`: có `name`, tham chiếu tới `FileSystemItem target`. Khi in: `Shortcut: <name> -> <targetPath>`.
9. `targetPath` có thể là đường dẫn logic theo dạng `/root/docs/a.txt`.
10. `Folder`: có `name` và danh sách `FileSystemItem` con. Khi in: `Folder: <name>` rồi in tiếp từng phần tử con với indent tăng thêm.
11. Trong `main`, tạo cây thư mục có ít nhất 2 cấp và đủ 3 loại phần tử, sau đó gọi `print("")`.
12. Ví dụ output (tham khảo):

```
Folder: root
  Folder: docs
    File: a.txt (12KB)
    File: b.txt (8KB)
    Shortcut: a-shortcut -> /root/docs/a.txt
  File: readme.md (4KB)
```

Bài 2: Hệ thống gửi thông báo đa kênh

Xây dựng hệ thống gửi thông báo theo mẫu Decorator.

Yêu cầu:

1. Tạo interface `Notifier` với `send(String msg)`.
2. Cài đặt `EmailNotifier` là kênh mặc định.
3. Tạo lớp trừu tượng `NotifierDecorator` giữ một `Notifier` và chuyển tiếp `send`.
4. Tạo ít nhất 2 decorator: `SMSNotifier`, `FacebookNotifier` (mỗi lớp gửi thêm 1 kênh).
5. Trong `main`, tạo `Notifier` và kết hợp ít nhất 2 decorator, ví dụ: Email + Facebook + SMS.
6. Khi gọi `send`, phải in đầy đủ các kênh đã gắn.

Bài 3: Hệ thống định dạng báo cáo

Bạn có đoạn mã ban đầu (đã vi phạm SRP và OCP):

```
class ReportService {  
    public String export(String type, Report data) {  
        if ("JSON".equalsIgnoreCase(type)) { /* ... */ }  
        else if ("XML".equalsIgnoreCase(type)) { /* ... */ }  
        else return "";  
    }  
}
```

Yêu cầu:

5. Tách trách nhiệm: `ReportService` không biết chi tiết định dạng.
6. Thiết kế theo OCP: thêm định dạng mới mà không sửa `ReportService`.
7. Gợi ý: dùng interface `ReportFormatter`.
8. Cài đặt lớp `Report` gồm `title`, `content` và getter.
9. Cài đặt ít nhất 2 formatter: `JsonFormatter`, `XmlFormatter`.
10. `ReportService` nhận `ReportFormatter` qua constructor và có `export(Report data)`.

11. `main` tạo `Report`, chọn formatter, gọi `export` và in kết quả.

Bài 4: Trình phát đa phương tiện

Xây dựng hệ thống phát media áp dụng DIP và ISP.

Yêu cầu:

1. Tạo interface nhỏ cho từng chức năng:
 - `AudioPlayable` có `playAudio(String file)`.
 - `VideoPlayable` có `playVideo(String file)`.
2. Tạo lớp `AudioPlayer` chỉ implement `AudioPlayable`.
3. Tạo lớp `VideoPlayer` chỉ implement `VideoPlayable`.
4. Tạo lớp `MediaPlayer` nhận phụ thuộc qua constructor:
 - `MediaPlayer(AudioPlayable audio, VideoPlayable video)`
 - Không được tạo trực tiếp `new AudioPlayer()` hoặc `new VideoPlayer()` bên trong `MediaPlayer`.
5. `main` tạo các player cụ thể rồi truyền vào `MediaPlayer`, sau đó gọi `playAudio` và `playVideo`.

Bài 5: Hệ thống ghi log – áp dụng Singleton Pattern

Mục tiêu

- Thiết kế một hệ thống ghi log đảm bảo chỉ tồn tại duy nhất một đối tượng `Logger` trong toàn bộ chương trình.
- Áp dụng mẫu thiết kế Singleton.

Yêu cầu

- Tạo lớp `Logger`:
 - Có thuộc tính: `private static Logger instance`.
 - Constructor phải là `private`.
 - Cung cấp phương thức: `public static Logger getInstance()` để trả về đối tượng duy nhất.
 - Nếu instance chưa tồn tại thì tạo mới (lazy initialization).
- `Logger` có các phương thức:

- void logInfo(String msg)
 - void logError(String msg)
- Khi gọi, in ra màn hình theo format:
 - [INFO] message
 - [ERROR] message
- Trong main:
 - Gọi Logger.getInstance() ở nhiều nơi.
 - Kiểm tra hai biến logger có cùng địa chỉ không.
 - Ghi nhiều log khác nhau.
- Không được:
 - Tạo đối tượng Logger bằng new Logger().
 - Tạo nhiều instance.

Output tham khảo

Logger instances equal: true

[INFO] Application started

[INFO] Processing data...

[ERROR] Something went wrong